AI Applications in Research

The Case of Claude Code as a Research Workflow

Compiled summary

June 10, 2026

Abstract

This document summarizes the main concepts surrounding the use of artificial intelligence in empirical research, with particular emphasis on *agentic coding assistants*—and Anthropic's **Claude Code** in particular—as instruments for conducting reproducible quantitative work. It synthesizes recurring themes from Scott Cunningham's ongoing "Claude Code" series on *Scott's Mixtape Substack*, including the most recent installment, "Claude Code 54: Amnesia" (June 2026). The aim is to provide a compact conceptual map for researchers considering, or already using, AI agents in their empirical pipelines.

Contents

1 Introduction: From Autocomplete to Agentic Research **1**

2 Core Concepts **1**
2.1 The Research Workflow as Software 1
2.2 What an AI Agent Contributes 2
2.3 Dialogue-Driven (Conversational) Research 2

3 The Central Challenge: "Drift" **2**
3.1 Why Drift Is Hard to Detect 2
3.2 Drift as a New Form of an Old Problem 3

4 Toward Solutions: Building a "Harness" **3**

5 Broader Themes for AI in Research **3**

6 Practical Recommendations **4**

7 Conclusion **4**

1 Introduction: From Autocomplete to Agentic Research

Artificial intelligence in research has moved well beyond passive text generation. The current frontier is the *agentic* assistant: a system that can read and write files, execute code, run analyses, manage version control, and iterate toward a goal through extended dialogue. **Claude Code** is a prominent example—a command-line agent that operates directly inside a researcher's project directory.

For empirical disciplines such as economics, the appeal is immediate. The craft of applied work has always rested on disciplined software engineering habits that most researchers acquire slowly and imperfectly. An agentic assistant can supply those habits by default.

2 Core Concepts

2.1 The Research Workflow as Software

Modern empirical research is, in practice, a software pipeline: raw data flow through cleaning, transformation, estimation, and the automated production of tables and figures. The quality of a project is largely the quality of this pipeline. Key disciplines include:

- **Consistent naming conventions** for files and folders.
- **A well-organized, repeatable directory structure** reused across projects.
- **Automated production** of tables and figures (no manual copy-paste).
- **A pipeline of code** rather than disconnected one-off scripts.
- **Version control** (e.g., `git`) for history and reproducibility.

2.2 What an AI Agent Contributes

The central observation is that a capable coding agent performs the above disciplines *natively*. Claude Code creates replicable code, organizes the directory sensibly, generates figures and tables programmatically, applies consistent naming, and—because git operations are accessible to it—makes version control straightforward. For a researcher, this can lower the activation energy of good practice and, plausibly, reduce the rate of mechanical error.

2.3 Dialogue-Driven (Conversational) Research

A distinctive mode of use is *co-authoring the research process in real time* through conversation, rather than dictating discrete tasks or designing rigid multi-step orchestration in advance. The researcher "talks with and to" the agent. This is powerful for discovery and exploration, but—as discussed below—it shifts where and how errors are produced.

3 The Central Challenge: "Drift"

Definition: Drift

Drift is the subtle, often unnoticed degradation of an empirical project over time when work is mediated by an AI agent. The project's foundations move without the researcher's awareness, and the resulting errors appear *outside* the locations where an experienced researcher is trained to look.

Concrete manifestations of drift include:

- **Phantom scripts:** An analysis step is performed ad hoc during dialogue but never written down as a durable, re-runnable script.
- **Shifting analytical sample:** The sample silently changes over time, implying a script is being modified unknowingly—or that the project's foundation is coming loose.

- **Mutating treatment definitions:** Treatment units subtly differ from how they were defined weeks or months earlier.
- **Stale exhibits:** A proliferation of figures and tables, some of which have quietly become outdated, with no easy way to tell which.

3.1 Why Drift Is Hard to Detect

The danger is not that errors occur—errors always occur—but that they are *not caught in real time*. Two structural reasons stand out:

1. **The "well-designed sprawl" problem.** An agent's output does not look like a mess. It looks organized—"a Claude mess, not a human mess"—which makes it harder to recognize, diagnose, and repair. The familiar visual cues of disorder are absent.
2. **The erosion of memorization.** Researchers historically compensate for fallible memory through *repetition and hyper-focus*: doing the same steps repeatedly builds an internal map of the project. When the agent writes the scaffolding instead, that compensating memorization never forms. Returning to a project after even a couple of days can leave one "unusually amnesiac."

3.2 Drift as a New Form of an Old Problem

Drift is recognizably a descendant of pre-AI pathologies—for example, the "poor man's version control" of saving ten date-stamped copies of a file. The difference is that the AI-era version is camouflaged by tidiness, making it *harder* to notice than its messy human ancestor.

4 Toward Solutions: Building a "Harness"

The proposed response is not a single feature but a *system*—a personal infrastructure, or **harness**, that is the physical expression and capture of the research workflow. Several design principles emerge:

- **Simple, habitual policies over clever tooling.** The most promising fix is a small set (roughly 5–6) of policy rules that, when *habitually* followed, make errors *knowable and controllable*. A bespoke "skill" is viewed skeptically because the problem is cognitive and habitual, not merely technical.
- **Progress logs.** The agent writes a log after every major session, creating a durable trail of what changed and why.
- **The "beautiful deck."** A daily process reads the progress logs and generates a flippable HTML slide deck—with beautiful figures, tables, and slides—accessible from a dashboard side panel. Its sole purpose is to *re-orient* the researcher after time away: a deliberate antidote to amnesia.
- **External mapping (analog notebook).** A large analog notebook to map what the agent is doing, acknowledging that the agent currently has "too much latitude."

Design Principle

Build systems *endogenous* to your constraints. Rather than assuming away human limitations (memory, attention, fallibility), design the workflow so that those limitations are

explicitly accommodated. Acceptance of the constraint is the precondition for a true fix rather than a band-aid.

5 Broader Themes for AI in Research

- **Reproducibility is necessary but not sufficient.** An agent can make code reproducible while the *conceptual* foundation (sample, treatment, identification) still drifts. Reproducibility of a moving target is not enough.
- **Agent latitude vs. control.** The more autonomy the agent has, the greater the productivity *and* the greater the surface for silent error. Effective use requires deliberate constraints on that latitude.
- **Human capital must adapt.** Researchers' error-detection instincts were trained on human-generated workflows; AI shifts the distribution of errors and requires new monitoring habits.
- **Disclosure and norms.** The series also engages with the professional dimension—how journals and the field should handle disclosure of AI use—underscoring that adoption is as much a normative question as a technical one.

6 Practical Recommendations

Distilling the above into actionable guidance for researchers using Claude Code or similar agents:

1. **Insist on durable scripts.** Require that every analysis step be written to a re-runnable script, never left as ephemeral dialogue.
2. **Pin the foundations.** Define and freeze the analytical sample and treatment definitions explicitly; audit them whenever the project resumes.
3. **Keep a session log.** Have the agent record what changed after each session, and review it on return.
4. **Maintain a re-orientation artifact.** Use a dashboard, deck, or notebook whose only job is to rebuild your mental model after time away.
5. **Adopt a few non-negotiable habits.** A short, fixed checklist beats elaborate tooling; consistency is what makes errors visible.
6. **Periodically rebuild from the ground up.** When drift is suspected, reconstructing a project from its foundations can be more honest than patching.

7 Conclusion

Agentic AI assistants like Claude Code can deliver, by default, the engineering discipline that empirical research has always demanded—reproducible code, organized projects, automated exhibits, and accessible version control. Yet the same fluency introduces a new and subtle failure mode, *drift*, in which a project's conceptual foundations move without notice precisely because the agent's output looks orderly. The path forward is not to abandon the tool but to wrap it in a deliberate *harness*: simple habitual policies, durable logs, re-orientation artifacts, and explicit constraints on agent

latitude. The guiding philosophy is to build systems *endogenous* to human limitations—designing for the researcher one actually is, not the one a tool pretends them to be.

Source material: Scott Cunningham, "Claude Code" series, *Scott's Mixtape Substack*, including "Claude Code 54: Amnesia" (June 10, 2026) and related installments on harnessing AI agents for economic research, AI disclosure norms, and reproducible empirical workflows.